

Fusion Lab Studio — T3 інтеграції DeepSeek Harness + Type-Safe Jev

Версія: 1.0 · **Дата:** 24.09.2026 · **Статус:** проєкт для coding-сесії

Модуль: Fusion Living Characters / Magic Characters

Мова інтерфейсу: українська; підтримка багатомовних книг.

1. Мета

Реалізувати режим AI-співавторства: письменник створює персонажів і сюжетні рамки, позначає одного чи кількох героїв як **AI-персонажів**, а система генерує їхні репліки, поведінку, рішення та взаємодію. Автор може передати AI окремий діалог, допит або цілу сцену. Опційний режим «**Таємниця від автора**» дозволяє агенту мати заздалегідь зафіксовані приховані мотиви, які розкриваються за сюжетними умовами.

Ключовий принцип: **Jev оцінює підготовлений структурований стан і вибирає серед допустимих альтернатив; LLM формує варіанти, пише прозу й узагальнює книгу; Harness оркеструє агентів та інструменти; Story Core забезпечує факти, права доступу та канон.** Не припускати, що Jev самостійно читає необмежений рукопис або генерує художній текст.

2. Офіційні джерела та обмеження

- DeepSeek Harness: <https://github.com/deepseek-ai/deepseek-harness> — архітектура everything-is-a-plugin; статус **Developer Preview**, можливі несумісні зміни.
- Запуск Harness для локального тесту: `npx @deepseek-ai/dsh web` (типово 127.0.0.1:3080).
- TypeSafe Jev: <https://docs.typesafe.ai/introduction> — структуровані питання **Choice, Score, Noul**. Choice і Score повертають також розподіл і confidence; кілька незалежних питань можуть оцінювати один стан.
- Конкретні SDK-методи, назви пакетів і payload API **не зафіксовані цим T3**: під час реалізації перевірити поточну документацію та ізолювати версійні відмінності в адаптерах.
- Значення Jev не є психологічним діагнозом або доказом істинності факту у світі книги; це допоміжні модельні оцінки.

3. Межі MVP

У MVP: 1-3 AI-персонажі в сцені; повна індексація згадок; персональна пам'ять; Jev Choice/Score; LLM-генерація реплік; режисер сцени; ручне затвердження канону; ізоляція секретів; інтерфейс «Магічна кнопка» та «Допит персонажа».

Поза MVP: необмежена кількість одночасних агентів, повністю автономне написання цілої книги, гарантована сюжетна якість без редактора, навчання власної моделі Jev, безпечне приховування секрету від адміністратора інфраструктури.

4. Архітектура

flowchart TD

```
UI[Writer UI / Magic Characters] --> API[Fusion API]
API --> CORE[Story Core: PostgreSQL]
API --> ORCH[Fusion AI Orchestrator]
CORE --> RET[Hybrid Retrieval: full-text + vector + graph]
RET --> PROFILE[Character Profile Builder / LLM]
PROFILE --> JEV[Jev Decision Adapter]
JEV --> ORCH
ORCH --> H[DeepSeek Harness Adapter]
```

```

H --> DIRECTOR[Scene Director]
DIRECTOR --> C1[Character Agent A]
DIRECTOR --> C2[Character Agent B]
DIRECTOR --> C3[Character Agent C]
C1 --> WRITER[Scene Writer LLM]
C2 --> WRITER
C3 --> WRITER
WRITER --> CHECK[Continuity & Knowledge Check]
CHECK --> DRAFT[Isolated Draft / Simulation]
DRAFT --> APPROVAL[Author Review]
APPROVAL -->|approve| CORE

```

Правило: UI звертається лише до Fusion API. Секретні дані та ключі Jev/LLM/Harness не потрапляють у браузер. Harness і Jev підключаються через адаптери, а не напряму до редактора.

4.1 Компоненти

1. **Retrieval Service:** усі згадки character_id, його діалоги, події, стосунки, часові стани та джерела; інкрементальне оновлення після правок.
2. **Profile Builder LLM:** стискає джерела в доказовий профіль, розрізняє confirmed, suggested, contradicted, unknown; не додає вигадані факти до канону.
3. **Jev Adapter:** оцінює атомарні запитання за коротким релевантним станом, повертає нормалізований результат.
4. **Harness Adapter:** запускає агентів і tools; контролює тайм-аути, ліміти, права та ізоляцію контекстів.
5. **Director:** визначає порядок ходів, межі сцени, умови завершення й дозволені сюжетні зміни.
6. **Character Agent:** отримує тільки персонально дозволені факти, обирає дії з Jev та формує намір репліки.
7. **Scene Writer LLM:** перетворює протокол симуляції на цілісну прозу відповідно до стилю автора.
8. **Continuity Checker:** перевіряє часові, сюжетні та епістемічні суперечності.
9. **Secret Vault:** зберігає приховані мотиви, плани й умови розкриття, не включаючи їх до загальних індексів.

5. Семантична модель персонажа

Використовувати існуючі типи Story Core: /character, /scene, /event, /emotion, /goal, /need, /belief, /relationship, /decision, /threshold, /dialogue, /character-state, /storyline, /location, /mystery, /story-promise.

Для кожного висновку зберігати: entity_id, source_paragraph_ids, source_revision, valid_story_time, status, visibility, created_by, updated_at. Згадки і зв'язки зберігати окремо; псевдоніми персонажів резолвити в один UUID.

5.1 Шари пам'яті

- **Public canon:** підтверджені автором факти, доступні за правилами ролей.
- **Character belief:** те, що герой вважає правдою, навіть якщо помиляється.
- **Character knowledge:** події та відомості, які герой міг отримати до поточної сцени.
- **Private motives:** приховані цілі й плани, доступні лише власному агенту та режисеру за необхідності.
- **Author-hidden secrets:** зафіксовані до симуляції секрети з умовами розкриття.
- **Simulation state:** тимчасові рішення й емоції поточного прогону, які ще не стали каноном.

Відокремлювати world_truth, character_belief та reader_knowledge. Не розкривати персонажу секрет іншого героя через спільний prompt.

6. Jev: оцінювання перед генерацією

6.1 Підготовка стану

Jev отримує **обмежений структурований snapshot**, а не весь рукопис: поточну мету, страхи, переконання, доступні факти, стосунки, ситуацію, допустимі дії, сценарні заборони та посилання на докази. Якщо даних замало, спершу запустити Retrieval + Profile Builder.

6.2 Атомарні питання

- **Choice:** «Яку дію обере персонаж із цього дозволеного списку: відповісти, збрехати, промовчати, змінити тему?»
- **Score:** «Наскільки сильний поточний страх персонажа за рубрикою 0–10?»
- **Score:** «Наскільки ця дія відповідає встановленому стилю поведінки?»
- **Noul:** «Чи узгоджується твердження X з наданим доказовим snapshot?» — результат лише допоміжний, не автоматичне підтвердження канону.

Не ставити одне надскладне питання «Оціни всього персонажа». Декомпонувати на незалежні виміри й агрегувати в коді за явною конфігурацією жанру/сцени. confidence використовувати для маршрутизації на повторну перевірку або авторське рішення; не трактувати як статистичну гарантію.

6.3 Внутрішній контракт адаптера (псевдосхема, НЕ офіційний Jev API)

```
{
  "character_id": "uuid",
  "scene_id": "uuid",
  "state": {
    "goal": "protect_anna",
    "fear": 8,
    "trust_in_investigator": 3,
    "known_facts": ["fact-1"],
    "allowed_actions": ["answer", "lie", "silence", "deflect"]
  },
  "questions": [
    { "id": "next_action", "kind": "choice", "options": ["answer", "lie", "silence", "deflect"] },
    { "id": "fear_intensity", "kind": "score", "rubric": "0-10" }
  ]
}
```

Нормалізована відповідь нашого адаптера: selected_action, scores, raw_distributions, confidence, model_version, snapshot_hash, decision_trace_id. Перевіряти допустимість обраної дії серверним валідатором; якщо порушено обмеження, повернутися до альтернативи або запросити рішення режисера.

7. Harness: агенти, skills, tools

7.1 Агенти

- character-profile-agent — збір та інтерпретація біографії.
- character-agent — ізольована роль одного героя, окрема пам'ять та knowledge boundary.
- scene-director-agent — хід сцени, сюжетні обмеження, контроль темпу.
- scene-writer-agent — літературний текст із протоколу дій.
- continuity-agent — перевірка знань, часу, причинності й канону.

- secret-curator-agent — одноразове створення таємниць і умов розкриття.

7.2 Skills (власні файли Fusion, не твердження про готові skills у Harness)

```
ai/
  adapters/
    harness/
      jev/
      llm/
  agents/
    character-profile/
    character/
    scene-director/
    scene-writer/
    continuity/
    secret-curator/
  skills/
    character-arc/SKILL.md
    dialogue-craft/SKILL.md
    emotion-dynamics/SKILL.md
    interrogation/SKILL.md
    mystery-foreshadowing/SKILL.md
    author-style/SKILL.md
    continuity-check/SKILL.md
  tools/
    get-character-snapshot/
    get-scene-context/
    search-character-mentions/
    evaluate-character-options/
    read-authorized-secret/
    write-simulation-event/
    propose-canon-change/
  contracts/
    character-snapshot.schema.json
    decision-result.schema.json
    scene-simulation.schema.json
```

7.3 Безпека tools

Читання й запис — тільки через серверні tools із project_id, actor_id, character_id, scene_id, simulation_id. Інструменти зміни канону заборонені агентам без підтвердження автора. Заборонити довільний filesystem/shell/network доступ у production-сценаріях книги. Валідувати всі результати моделей за схемами.

8. Протокол Magic Scene

1. Автор вибирає 1-3 персонажів, позначає AI Character, встановлює мету, жанр, довжину, темп, дозволені зміни та секретність.
2. Система фіксує book_revision і створює simulation_id.
3. Retrieval формує окремий snapshot кожного героя **станом на момент сцени**, без майбутніх спойлерів.
4. Profile Builder визначає підтверджені риси, суперечності та невідомі дані.
5. Director пропонує обмежений перелік допустимих дій активного героя.
6. Jev оцінює атомарні характеристики й вибір; серверний валідатор застосовує жорсткі заборони.

7. Character Agent генерує **намір**, репліку та дію, виходячи тільки зі свого дозволеного контексту.
8. Director передає іншим агентам тільки спостережувані події; приховані думки та секрети не транслюються.
9. Кроки 5–8 повторюються до мети сцени, ліміту ходів, токенів, часу або команди автора «Стоп».
10. Scene Writer перетворює протокол на чернетку. Continuity Checker створює попередження з доказами.
11. Автор обирає: Прийняти, Частково прийняти, Перегенерувати, Відхилити, Продовжити сцену.
12. Лише затверджені зміни входять до канону; решта залишаються ізольованою симуляцією.

Режим прискорення: кешувати профілі за `character_id + book_revision + story_time`; повторно оцінювати Jev лише змінені параметри; паралельно готувати незалежні профілі, але репліки виконувати послідовно за ходами. Використовувати `streaming` для чернетки, не обходячи перевірку перед записом у канон.

9. Режим «Таємниця від автора»

- Автор дозволяє `hidden_secret_mode` для конкретного персонажа і задає межі: теми, жанр, недопустимі повороти, найраніший акт розкриття.
- `secret-curator-agent` генерує секрет **до** відповідної симуляції; сховище записує версію, часову позначку, хеш і незмінний журнал створення.
- У звичайному UI автор бачить лише Секрет створено, рівень ризику для сюжету та можливі умови розкриття без змісту.
- Агент власника секрету отримує лише необхідні приватні факти; інші агенти бачать лише спостережувані наслідки.
- Режисер перевіряє, що розкриття не суперечить підтвердженому канону; до моменту розкриття допускаються натяки, але не ретроспективне переписування секрету.
- Передбачити Emergency Reveal для автора, Freeze Secret, Regenerate Before First Use, Archive Simulation.
- Ізоляція має бути **серверною**, не лише приховуванням елемента UI. Не включати секрети в загальний векторний пошук, клієнтські журнали та зовнішню телеметрію.
- Чесно повідомляти автору: таємниця прихована від звичайного авторського інтерфейсу, але технічні адміністратори з належними правами можуть мати доступ до інфраструктури.

10. Схема даних — нові таблиці

```

character_agents(id, project_id, character_id, enabled, autonomy_level,
                 agent_config_json, model_policy_json, created_at)
character_memories(id, project_id, character_id, memory_type, content,
                  story_time, evidence_json, visibility, status, revision)
character_secrets(id, project_id, owner_character_id, encrypted_payload,
                 reveal_policy_json, status, commitment_hash, created_at)
character_states(id, project_id, character_id, scene_id, simulation_id,
                 goals_json, emotions_json, beliefs_json, relationships_json,
                 state_version)
character_decisions(id, project_id, simulation_id, character_id, turn_index,
                   options_json, jev_result_json, selected_action,
                   validation_json, snapshot_hash, created_at)
scene_simulations(id, project_id, scene_id, base_book_revision,
                  config_json, status, current_turn, created_by, created_at)
simulation_events(id, simulation_id, turn_index, actor_character_id,
                 event_type, public_payload_json, private_payload_ref,
                 source_decision_id, created_at)
canon_proposals(id, project_id, simulation_id, source_event_ids,

```

proposed_change_json, status, reviewed_by, reviewed_at)

Усі таблиці: FK, індекси за project_id, RBAC/RLS, контроль версій і міграції. Секретні payload шифрувати, ключі зберігати поза БД у відповідному secret manager. Журнал аудиту не повинен дублювати прихований зміст.

11. API Fusion (власні маршрути, не API Harness/Jev)

Method	Route	Призначення
POST	/api/projects/:id/characters/:id/agent	Увімкнути AI-персонажа
GET	/api/projects/:id/characters/:id/agent	Додати до списку профіль
POST	/api/projects/:id/characters/:id/agent	Почати дію персонажа
POST	/api/projects/:id/scenes/:id/scene	Створити Magic Scene
POST	/api/simulations/:id/step	Наступний хід
POST	/api/simulations/:id/pause	Призупинити
GET	/api/simulations/:id/events	Події без недозволених секретів
POST	/api/simulations/:id/draft	Створити літературну чернетку
POST	/api/simulations/:id/approve	Затвердити вибрані зміни
POST	/api/characters/:id/secrets/:id/reveal	Аварійне розкриття з підтвердженням

Довгі операції запускати через job queue; підтримати job_id, progress, cancel, retry, idempotency key, rate limit і бюджет токенів/запитів на проєкт.

12. UI

Картка персонажа: перемикач ☐ AI-персонаж; рівень автономності; «Допитати»; «Додати до Magic Scene»; доступний профіль; стан пам'яті; індикатор прихованого секрету.

Редактор сцени: учасники з позначкою AI/Author; мета, конфлікт, обмеження, кількість ходів; кнопки , , , , ; роздільні вкладки «Протокол агентів» і «Літературний текст».

Перевірка чернетки: diff до поточного рукопису, попередження continuity, джерела рішень, кнопки затвердження кожного фрагмента, заборона автоматичного перезапису без явного вибору автора.

Таємниця: індикатор «Прихований мотив активний», налаштування умов розкриття, аварійне розкриття з додатковим підтвердженням.

13. Політики якості та безпеки

- **No spoiler leakage:** агент бачить лише знання, які мав на момент сцени.
- **No retroactive secrets:** використаний секрет не змінювати заднім числом.
- **No canon without approval:** модель не змінює підтверджені факти самостійно.
- **Provenance:** кожен висновок про характер має джерело або позначку generated_hypothesis.
- **Fail closed:** за помилки Jev/LLM/ACL не розкривати секрет і не виконувати запис у канон.
- **Fallback:** за недоступності Jev — налаштований LLM structured-output adapter або запит рішення автора; чітко позначити режим у журналі.
- **Version pinning:** зафіксувати перевірені версії Harness, Jev SDK/API та LLM-провайдера; контрактні тести після оновлень.
- **Observability:** latency, tokens, cost, Jev calls, fallback rate, continuity violations, rejected decisions; приватний вміст не логувати за замовчуванням.

14. Етапи реалізації

Phase 0 — Spike (підтвердження сумісності)

- Перевірити актуальні Harness plugins/SDK і Jev API за офіційною документацією.
- Підняти локальний Harness, виконати один tool call, один Jev Choice та один LLM-виклик через наші адаптери.
- Зафіксувати версії, схеми та обмеження провайдерів.

Phase 1 — Один живий персонаж

- Індекс згадок і профіль із доказами; AI-перемикач; Jev Decision Adapter; 1 агент; допит; ручне затвердження.

Phase 2 — Двоє/троє персонажів

- Scene Director, ізоляція знань, покрокова симуляція, генератор літературної сцени, continuity, diff.

Phase 3 — Приховані секрети

- Secret Vault, immutable commitment, reveal policies, натяки, аварійне розкриття, тести витоків.

Phase 4 — Оптимізація

- Кеш snapshot, інкрементальний аналіз, паралельна підготовка, streaming, бюджети й метрики якості.

15. Критерії приймання

1. Для персонажа, позначеного AI, система знаходить усі доступні згадки й показує джерела сформованого профілю.
2. Jev оцінює щонайменше Choice і Score на підготовленому snapshot; адаптер повертає нормалізований результат і версію моделі.
3. Один AI-персонаж може відповісти на запитання автора без зміни канону.
4. Два AI-персонажі проводять не менше 10 контрольованих ходів; третій може бути доданий до сцени.
5. Персонаж не використовує відомостей, отриманих у пізніших главах або з приватної пам'яті іншого героя.
6. Прихований секрет не з'являється у звичайному UI, загальному пошуку, API для іншого персонажа та доступних автору логах до виконання умови розкриття.
7. Повторна генерація не змінює раніше зафіксований секрет; є журнал версій і hash commitment.
8. Автор може зупинити симуляцію, переглянути diff і затвердити лише вибрані фрагменти.
9. За збою Jev система переходить у задекларований fallback без втрати стану чи витоків секретів.
10. Оновлення версії Harness не потребує змін UI або Story Core завдяки адаптеру.

16. Перший coding task

1. Створити ai/adapters/harness, ai/adapters/jev, ai/contracts.
2. Реалізувати CharacterSnapshotBuilder на основі існуючих entities, entity_mentions, entity_relations.
3. Створити внутрішній метод evaluate_character_turn(snapshot, allowed_actions) і mock Jev adapter для тестів.

4. Додати одну кнопку □ AI-персонаж та маршрут створення симуляції.
 5. Запустити наскрізний тест: **картка персонажа → retrieval → Jev Choice → Harness character agent → репліка → чернетка → затвердження.**
-

Архітектурне рішення: Jev не замінює Story Core чи LLM. Він є спеціалізованим Decision Engine всередині багатокрокового процесу, який оркеструє Harness. Це дозволяє змінювати модель оцінювання або генеративну LLM незалежно від книги, UI та канону.